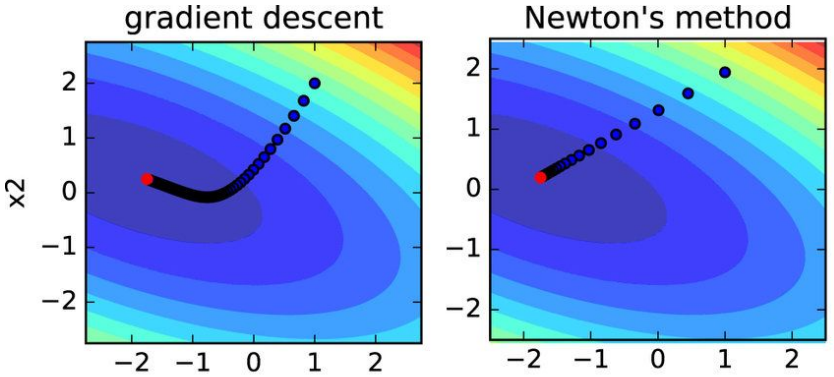
LBFGS

Грек Фёдор

Мотивация. Зачем это нужно?

Метод	Градиентный спуск	Метод Ньютона
Скорость сходимости	медленная	быстрая
Время на одну операцию	O(n)	O(n^3), взятие обратной матрицы



Градиентный спуск

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

Метод Ньютона

$$x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

BFGS

BFGS - это квазиньютоновский метод, который использует приближение обратного гессиана

Алгоритм

Given starting point x_0, convergence tolerance $\epsilon > 0$,
    inverse Hessian approximation H_0;
$k \leftarrow 0$;
while $\|\nabla f_k\| > \epsilon$;
    Compute search direction

$$p_k = -H_k \nabla f_k;$$

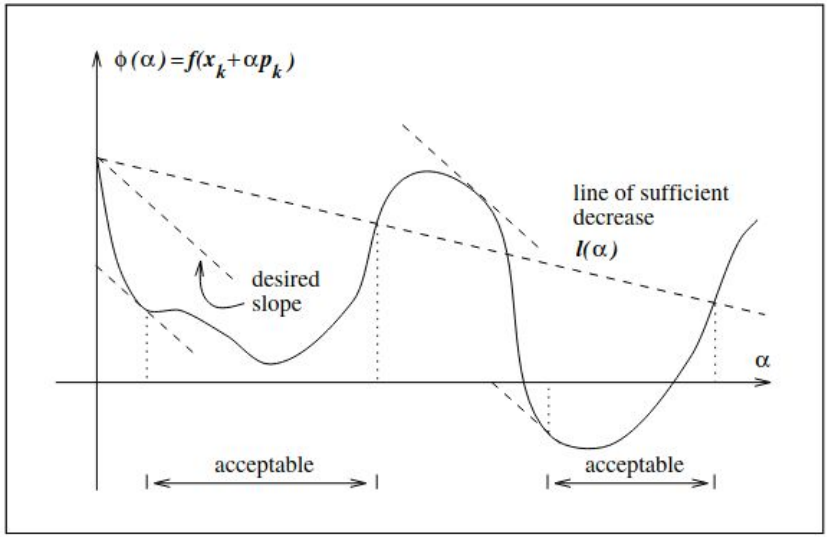
    Set $x_{k+1} = x_k + \alpha_k p_k$ where α_k is computed from a line search
        procedure to satisfy the Wolfe conditions
    Define $s_k = x_{k+1} - x_k$ and $y_k = \nabla f_{k+1} - \nabla f_k$;
    Compute H_{k+1}
    $k \leftarrow k + 1$;
end (while)

Wolfe conditions

$$f(x_k + \alpha_k p_k) \leq f(x_k) + c_1 \alpha_k \nabla f_k^T p_k, \tag{3.6a}$$

$$\nabla f(x_k + \alpha_k p_k)^T p_k \geq c_2 \nabla f_k^T p_k, \tag{3.6b}$$

with $0 < c_1 < c_2 < 1$.



BFGS. Вычисление H_{k+1}

symmetric and positive definite, and must satisfy the secant equation (6.6), now written as

$$H_{k+1} y_k = s_k.$$

The condition of closeness to H_k is now specified by the following analogue of (6.9):

$$\min_H \|H - H_k\| \tag{6.16a}$$

$$\text{subject to} \quad H = H^T, \qquad H y_k = s_k. \tag{6.16b}$$

$$\text{(BFGS)} \qquad H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T,$$

$$\rho_k = \frac{1}{y_k^T s_k}.$$

BFGS. В чем проблема?

Вычисление одной итерации O(n^2), хотя сходимость суперлинейная

Theorem 6.6.

Suppose that f is twice continuously differentiable and that the iterates generated by the BFGS algorithm converge to a minimizer x^* at which Assumption 6.2 holds. Suppose also that (6.52) holds. Then x_k converges to x^* at a superlinear rate.

LBFGS

Идея:

Хранить последние $m \approx 3 - 20$ пар (s_i, y_i), чтобы через них обновлять гессиан

Как мы обновлялись в BFGS?

For example, recall the quasi-Newton method $x_{k+1} = x_k - \alpha_k H_k \nabla f(x_k)$ with BFGS update:

$$H_k = V_{k-1}^\top H_{k-1} V_{k-1} + \rho_{k-1} s_{k-1} s_{k-1}^\top, \tag{1}$$

where

$$\rho_k = \frac{1}{s_k^\top y_k}, \qquad V_k = I - \rho_k y_k s_k^\top,$$
$$s_k = x_{k+1} - x_k, \qquad y_k = \nabla f(x_{k+1}) - \nabla f(x_k),$$

and the stepsize α_k satisfies WWC. The matrices B_k and H_k constructed by BFGS are often dense,

LBFGS

Раскроем m итераций вычисления H_k

Recall and expand the BFGS update:

$$
\begin{aligned}
\text{BFGS:} \quad H_k =& V_{k-1}^\top H_{k-1} V_{k-1} + \rho_{k-1} s_{k-1} s_{k-1}^\top \\
=& V_{k-1}^\top V_{k-2}^\top H_{k-2} V_{k-2} V_{k-1} + \rho_{k-2} V_{k-2} s_{k-2} s_{k-2}^\top V_{k-1} + \rho_{k-1} s_{k-1} s_{k-1}^\top \\
=& \left(V_{k-1}^\top V_{k-2}^\top \cdots V_{k-m}^\top\right) H_{k-m} \left(V_{k-m} V_{k-m+1} \cdots V_{k-1}\right) \\
&+ \rho_{k-m} \left(V_{k-1}^\top \cdots V_{k-m+1}^\top\right) s_{k-m} s_{k-m}^\top \left(V_{k-m+1} \cdots V_{k-1}\right) \\
&+ \rho_{k-m+1} \left(V_{k-1}^\top \cdots V_{k-m+2}^\top\right) s_{k-m+1} s_{k-m+1}^\top \left(V_{k-m+2} \cdots V_{k-1}\right) \\
&+ \cdots \\
&+ \rho_{k-2} V_{k-1}^\top s_{k-2} s_{k-2}^\top V_{k-1} \\
&+ \rho_{k-1} s_{k-1} s_{k-1}^\top.
\end{aligned}
$$

In L-BFGS, we replace H_{k-m} (a dense $d \times d$ matrix) with some sparse matrix H_k^0, e.g., a diagonal matrix. Thus, H_k can be constructed using the most recent $m \ll d$ pairs $\{s_i, y_i\}_{i=k-m}^{k-1}$. That is,

$$
\begin{aligned}
\text{L-BFGS:} \quad H_k =& \left(V_{k-1}^\top V_{k-2}^\top \cdots V_{k-m}^\top\right) H_k^0 \left(V_{k-m} V_{k-m+1} \cdots V_{k-1}\right) \\
&+ \rho_{k-m} \left(V_{k-1}^\top \cdots V_{k-m+1}^\top\right) s_{k-m} s_{k-m}^\top \left(V_{k-m+1} \cdots V_{k-1}\right) \\
&+ \rho_{k-m+1} \left(V_{k-1}^\top \cdots V_{k-m+2}^\top\right) s_{k-m+1} s_{k-m+1}^\top \left(V_{k-m+2} \cdots V_{k-1}\right) \\
&+ \cdots \\
&+ \rho_{k-1} s_{k-1} s_{k-1}^\top.
\end{aligned}
$$

LBFGS

Как вычислить за O(mn) $H_k \cdot \nabla f(x_k)$?

Algorithm 1 L-BFGS two-loop recursion

set $q = \nabla f(x_k)$ want to compute $H_k \cdot \nabla f(x_k)$

for $i = k-1, k-2, \ldots, k = m$ **do**:

$\alpha_i \leftarrow \rho_i s_i^\top q$

$q \leftarrow q - \alpha_i y_i$ // RHS$= q - \rho_i s_i^\top q y_i = \underbrace{\left(I - \rho_i y_i s_i^\top\right)}_{V_i} q$

$r = H_k^0 q$

for $i = k - m$ to $k-1$:

$\beta \leftarrow \rho_i y_i^\top r$

$r \leftarrow r + s_i(\alpha_i - \beta)$ // RHS$= r + \rho_i \alpha_i - \rho_i y_i^\top r s_i = \underbrace{\left(I - \rho_i s_i y_i^\top\right)}_{V_i^\top} r + \rho_i \alpha_i$

return r // which equals $H_k \nabla f(x_k)$

LBFGS

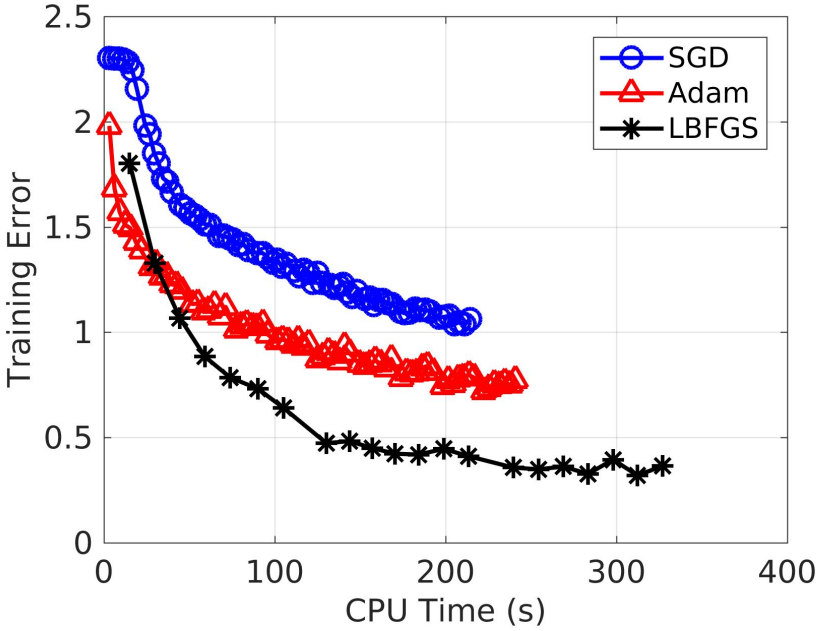
Algorithm 2 L-BFGS

input: $x_0 \in \mathbb{R}^d$ (initial point), $m > 0$ (memory budget), $\epsilon > 0$ (convergence criterion)
$k \leftarrow 0$
repeat:

- Choose H_k^0
- $p_k \leftarrow -H_k \nabla f(x_k)$, where $H_k \nabla f(x_k)$ is computed using Algorithm 1
- $x_{k+1} \leftarrow x_k + \alpha_k p_k$, where α_k satisfies Wolfe Conditions
- **if** $k > m$:
  - discard $\{s_{k-m}, y_{k-m}\}$ from storage
- Compute and store $s_k \leftarrow x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$
- $k \leftarrow k + 1$

until $\|\nabla f(x_k\| \leq \epsilon$

Пример с другими оптимизаторами



Итог

LBFGS по сложности итерации сравним с градиентным спуском, но зато значительно быстрее сходится.

Источники

1.https://www.researchgate.net/publication/311458101_Quantum_gradient_descent_and_Newton's_method_for_constrained_polynomial_optimization

2. https://habr.com/ru/articles/333356/

3. Jorge Nocedal, Stephen Wright. Numerical Optimization, Second Edition. Chapter 3, 6.
https://www.math.uci.edu/~qnie/Publications/NumericalOptimization.pdf

4. https://pages.cs.wisc.edu/~yudongchen/cs726_sp23/Lecture_23_L-BFGS.pdf

5. https://sagecal.sourceforge.net/pytorch/index.html

6.